

Java Collections Framework (JCF)

◆ Introduction

The **Java Collections Framework (JCF)** is a unified architecture for storing and manipulating groups of objects. It provides:

- Interfaces (List, Set, Map)
 - Implementations (ArrayList, HashSet, HashMap, etc.)
 - Algorithms (sorting, searching)
-

✚ 1. LIST

◆ Definition

A **List** is an ordered collection that allows duplicates and maintains insertion order.

✅ 1.1 ArrayList

✓ Idea

Dynamic array that resizes automatically.

✓ Features

- Allows duplicates
- Fast random access ($O(1)$)
- Slow insertion/deletion in middle

✓ When to Use?

- When you need fast access by index

✓ Important Methods

- `add()`
- `get()`
- `set()`
- `remove()`

- size()

✓ Example

```
import java.util.*;
```

```
public class Main {  
    public static void main(String[] args) {  
        ArrayList<String> list = new ArrayList<>();  
        list.add("Ali");  
        list.add("Ahmed");  
        list.add("Ali");  
  
        System.out.println(list);  
        System.out.println(list.get(1));  
    }  
}
```

✓ 1.2 LinkedList

✓ Idea

Doubly linked list (nodes connected with pointers)

✓ Features

- Fast insertion/deletion
- Slow random access

✓ When to Use?

- When frequent insert/delete operations

✓ Important Methods

- addFirst()
- addLast()
- remove()
- get()

✓ Example

```
LinkedList<Integer> list = new LinkedList<>();
```

```
list.add(10);
```

```
list.add(20);
```

```
list.addFirst(5);
```

```
System.out.println(list);
```

✓ 1.3 Vector

✓ Idea

Synchronized dynamic array (thread-safe)

✓ Features

- Thread-safe
- Slower than ArrayList

✓ When to Use?

- In multithreaded environment (legacy)

✓ Important Methods

- add()
- get()
- remove()

✓ Example

```
Vector<Integer> v = new Vector<>();
```

```
v.add(1);
```

```
v.add(2);
```

```
System.out.println(v);
```

✓ 1.4 Stack

✓ Idea

LIFO (Last In First Out)

✓ Features

- Extends Vector
- Works like stack

✓ When to Use?

- Undo operations, recursion

✓ Important Methods

- push()
- pop()
- peek()

✓ Example

```
Stack<Integer> stack = new Stack<>();
```

```
stack.push(10);
```

```
stack.push(20);
```

```
System.out.println(stack.pop());
```

2. SET

◆ Definition

A **Set** is a collection that does NOT allow duplicates.

✅ 2.1 HashSet

✓ Idea

Uses hash table

✓ Features

- No duplicates
- No order

✓ When to Use?

- When uniqueness matters

✓ Important Methods

- add()
- remove()
- contains()

✓ Example

```
HashSet<Integer> set = new HashSet<>();
```

```
set.add(1);
```

```
set.add(2);
```

```
set.add(1);
```

```
System.out.println(set);
```

✅ 2.2 LinkedHashSet

✓ Idea

HashSet + maintains insertion order

✓ Features

- No duplicates
- Ordered

✓ When to Use?

- Need uniqueness + order

✓ Example

```
LinkedHashSet<String> set = new LinkedHashSet<>();
```

```
set.add("A");
```

```
set.add("B");
```

```
System.out.println(set);
```

✓ 2.3 TreeSet

✓ Idea

Sorted set using Red-Black Tree

✓ Features

- Sorted
- No duplicates

✓ When to Use?

- Need sorted data

✓ Important Methods

- first()
- last()

- higher()

✓ Example

```
TreeSet<Integer> set = new TreeSet<>();
```

```
set.add(30);
```

```
set.add(10);
```

```
set.add(20);
```

```
System.out.println(set);
```

3. MAP

◆ Definition

A **Map** stores data in key-value pairs.

✓ 3.1 HashMap

✓ Idea

Hash table implementation

✓ Features

- Allows one null key
- No order

✓ When to Use?

- Fast lookup

✓ Important Methods

- put()
- get()
- remove()

- containsKey()

✓ Example

```
HashMap<Integer, String> map = new HashMap<>();
```

```
map.put(1, "Ali");
```

```
map.put(2, "Ahmed");
```

```
System.out.println(map.get(1));
```

✓ 3.2 LinkedHashMap

✓ Idea

HashMap + insertion order

✓ Features

- Ordered
- Faster than TreeMap

✓ When to Use?

- Need order + map

✓ Example

```
LinkedHashMap<Integer, String> map = new LinkedHashMap<>();
```

```
map.put(1, "A");
```

```
map.put(2, "B");
```

```
System.out.println(map);
```

✓ 3.3 TreeMap

✓ Idea

Sorted map using Red-Black Tree

✓ Features

- Sorted by keys

✓ When to Use?

- Need sorted keys

✓ Example

```
TreeMap<Integer, String> map = new TreeMap<>();
```

```
map.put(3, "C");
```

```
map.put(1, "A");
```

```
System.out.println(map);
```

✅ 3.4 Hashtable

✓ Idea

Synchronized HashMap (legacy)

✓ Features

- Thread-safe
- No null keys or values

✓ When to Use?

- Legacy systems

✓ Example

```
Hashtable<Integer, String> table = new Hashtable<>();
```

```
table.put(1, "Ali");
```

```
System.out.println(table);
```